

FLOXTO

# Final Pre-Launch Security, Privacy, Reliability, Performance & Code Quality Audit

Prepared from the current local codebase and Supabase SQL model in `c:\Users\user\Desktop\watstore`.

---

## A. Executive Summary

I audited the current Flocto codebase and SQL security model. I did not apply changes, because the meaningful issues are DB/RLS/permissions/dependency changes that the audit instructions said to report first.

**Launch decision: not ready for public production launch yet.**

Main blockers:

- Merchants may be able to self-change subscription fields through direct Supabase calls.
  - Public product RLS currently allows reading soft-deleted products.
  - `npm audit` reports high-severity dependency advisories, including `next`.
  - Security headers are not configured.
- 

## B. Launch Decision

**Do not launch until the Critical and High items below are fixed and verified in the live Supabase project.**

## C. Critical Findings

### CRITICAL

#### 1. Merchant subscription bypass risk

**Evidence:** broad owner update policy on merchants in `supabase/rls_policies.sql`.

**Risk:** a logged-in merchant could potentially update their own `subscription_status`, `billing_cycle_months`, `last_payment_date`, or `trial_start_date` directly via Supabase client tools, bypassing billing/trial enforcement.

**Fixed:** No, approval needed.

**Proposed fix:**

```
revoke update on public.merchants from authenticated;

grant update (
  business_name,
  tagline,
  delivery_info,
  payment_options,
  why_choose_us,
  slug,
  whatsapp_number,
  logo_url
) on public.merchants to authenticated;
```

Also update onboarding code to rely on DB defaults for trial fields rather than inserting subscription fields from the browser.

**CRITICAL****2. Merchant insert can set subscription fields**

**Evidence:** setup inserts subscription fields from the client in `app/dashboard/setup/page.tsx`.

**Risk:** direct client calls could insert an "active" merchant row if insert privileges are broad.

**Fixed:** No.

**Proposed fix:**

```
revoke insert on public.merchants from authenticated;

grant insert (
  user_id,
  business_name,
  tagline,
  delivery_info,
  payment_options,
  why_choose_us,
  slug,
  whatsapp_number,
  logo_url
) on public.merchants to authenticated;
```

**CRITICAL****3. Soft-deleted products are still publicly readable via direct API**

**Evidence:** public products policy is `using (true)` in `supabase/rls_policies.sql`.

**Risk:** storefront UI hides deleted products, but anonymous users could query soft-deleted product rows directly.

**Fixed:** No.

**Proposed fix:**

```
drop policy if exists "Anyone can read public products" on public.products;

create policy "Anyone can read public products"
on public.products
for select
to anon, authenticated
using (deleted_at is null);
```

## D. High Findings

**HIGH**

### 1. Dependency vulnerabilities

`npm audit --audit-level=low` reported six high-severity vulnerabilities, including `next`, `postcss`, `sharp`, `nanoid`, `js-yaml`, and `brace-expansion`.

**Fixed:** No.

**Reason:** `npm audit fix --force` would upgrade Next to `16.3.0`, outside the current declared version.

**Action:** test and upgrade dependencies before launch.

**HIGH**

### 2. Orders can be broadly updated/deleted by merchants

**Evidence:** order update/delete policies in `supabase/rls_policies.sql`.

**Risk:** a merchant cannot access another merchant's orders, but they may alter more than just `status`, or delete order history.

**Proposed fix:**

```
revoke update, delete on public.orders from authenticated;  
grant update (status) on public.orders to authenticated;  
drop policy if exists "Merchants can delete own orders" on public.orders;
```

**HIGH**

### 3. Merchants can delete their own merchant row

**Evidence:** delete policy in `supabase/rls_policies.sql`.

**Risk:** accidental or malicious self-deletion can break store/order continuity.

**Proposed fix:** remove merchant delete policy unless there is a real account deletion flow.

**HIGH****4. No security headers configured**

**Evidence:** `next.config.ts` has no headers.

**Risk:** weaker browser hardening.

**Proposed:** add `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, and a carefully tested CSP.

**HIGH****5. Public order RPC has no rate limiting**

**Evidence:** `create_public_order()` is executable by `anon`.

**Risk:** spam orders.

**Proposed:** add CAPTCHA/rate limiting via Edge Function or server-side route before public launch.

## E. Medium Findings

- Public storage buckets expose uploaded media permanently if someone has the URL.
- Storage policies do not appear to enforce MIME type.
- Service worker caches same-origin runtime GET responses network-first; Supabase data is cross-origin and not cached, but private route shells may still be cached.
- No live Supabase verification was possible from local code; the SQL file may differ from production.

## F. Low/Cleanup

- PWA manifest currently uses the tagline as `name` and `short_name`. This matches the splash-screen request, but is unusual for installed app labels.
- Some old default static assets may still exist if not used.
- `NEXT_PUBLIC_ADMIN_EMAIL` is only a UX shortcut, not real security. The real security correctly sits in the DB RPC.

## G. Passed Controls

- No `app/api`, route handlers, server actions, or middleware found.
- Auth pages use Supabase Auth client calls.
- Admin renewals page calls `get_admin_renewals()` instead of querying merchants directly.

- Product detail route checks both `productId` and `merchant_id`.
- Store/product UI queries filter `deleted_at is null`.
- Product delete UI uses soft delete.
- Order creation snapshots product name, sale price, and first photo.
- Root `/` redirects by auth/profile state.
- Password reset redirects to `/reset-password`.

## H. Supabase Table/RLS Matrix

| TABLE                            | STATUS                                                                    | RISK / NOTE                                                                             |
|----------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>merchants</code>           | RLS enabled; public SELECT policy plus owner select/insert/update/delete. | Needs narrowed insert/update/delete.                                                    |
| <code>products</code>            | RLS enabled; owner policies use <code>is_own_merchant()</code> .          | Public SELECT currently too broad because it does not exclude <code>deleted_at</code> . |
| <code>orders</code>              | RLS enabled; owner select/update/delete.                                  | Update should be status-only; delete should likely be removed.                          |
| <code>product_chat_clicks</code> | RLS enabled, table access revoked.                                        | RPC records/counts clicks.                                                              |
| <code>admin_users</code>         | RLS enabled, table access revoked.                                        | Used by admin RPC.                                                                      |

## I. Merchant Isolation

Cross-merchant isolation for products/orders is generally sound because policies use `public.is_own_merchant(merchant_id)`. The issue is not cross-merchant access; it is over-broad control of a merchant's own billing/order rows.

### J. `create_public_order()`

#### Good:

- Rejects missing quantity/name/location.
- Rejects deleted/missing/out-of-stock products.
- Refreshes subscription and blocks expired stores.
- Uses advisory lock for per-merchant order number.
- Stores product snapshots.

#### Concern:

- No rate limit or CAPTCHA.
- No SQL length limits for customer name/location.

## K. Admin Security

- `get_admin_renewals()` checks `public.admin_users` against `auth.uid()`.
- Client email check is only a shortcut.
- Manual action: confirm your actual Supabase `auth.users.id` is inserted into `public.admin_users`.

---

## L. Subscription Security

Trial/expiry calculation exists in SQL and app code. The critical gap is write permissions: merchants must not be able to directly write subscription columns.

---

## M. Storage/Upload

### Good:

- Product images compressed client-side to WebP.
- Product media paths are scoped by merchant ID.
- Logo paths are scoped by auth user ID.

**Concern:** add MIME/content-type enforcement in storage policies if possible.

---

## N. Secret/Git-History

Current tracked source scan found no service-role key, private DB URL, or obvious app secret. `.env.local` is not tracked.

---

## O. PWA/Cache

- Manifest and service worker exist.
- Cross-origin Supabase requests are not intercepted.
- Runtime pages use network-first.
- Consider excluding authenticated route shells from runtime caching for extra caution.



---

## P. Dependency Audit

Result: **failed due high vulnerabilities**. Needs dependency upgrade/testing before launch.

---

## Q. Security Headers

Missing. Add before launch.

---

## R. Privacy

Customer PII stored: `customer_name`, `delivery_location`, WhatsApp number. Add a privacy policy before real launch, and limit who can read orders to merchant/admin only.

---

## S. Performance

Build passes. Product images are compressed. Public pages avoid unnecessary complex features. Good baseline.

---

## T. Code Cleanup

No broad refactor recommended before launch. Fix the security items first.

---

## U. Manual Config Required

- Supabase Auth redirect URLs include `https://flocto.com/reset-password`.
  - Production env vars set: Supabase URL, anon key, site URL.
  - `admin_users` contains your exact auth UUID.
  - Run the SQL verification queries in production after policy changes.
- 

## V. Manual Pre-Launch Checklist

1. Apply approved RLS/permission fixes.
2. Upgrade vulnerable dependencies and rebuild.
3. Add security headers.
4. Test signup, setup, login, reset password.
5. Test expired trial and expired subscription.
6. Test public storefront/product pages.
7. Test order creation on Android and iPhone.
8. Test installed PWA after service worker update.

---

## W. Final Launch Blockers

1. Lock down merchant subscription insert/update permissions.
2. Change public products policy to exclude soft-deleted products.
3. Restrict order updates to `status` only and remove order delete unless needed.
4. Resolve high-severity npm audit findings.
5. Add security headers.
6. Verify live Supabase policies match the reviewed SQL.